

Synthetic Financial Time Series Generation

GAN Benchmark: TimeGAN · QuantGAN · FinGAN

BRICS Emerging Market Indices

Format	Dual-audience: Foundations (Part I) · Technical (Part II)
Collaboration	Universitat Politècnica de Catalunya (UPC)
Document type	Research draft — prepared for Quantitative Finance / ACM ICAIF
Models	TimeGAN (GRU) · QuantGAN (TCN-WGAN-GP) · FinGAN (CNN-WGAN-GP)
Markets	BOVESPA · FTSE JSE · MOEX · NIFTY50 · SHANGHAI
Generated	2026-09-07

Provenance — every number in this report is read from the run below, not hand-typed.

Generated	2026-09-06T19:33:08 (git 582af17a9b)
Seeds	42, 43, 44
Markets	BOVESPA, FTSE, MOEX, NIFTY50, SHANGHAI
Walk-forward folds	5
Generator updates	9,000
Software	Python 3.12.3 · PyTorch 2.13.0+cu130
CUDA	yes (NVIDIA GeForce RTX 4090)
Platform	Linux-6.8.0-124-generic-x86_64-with-glibc2.39

Summary — For All Readers

Non-specialist language · ≤ 200 words

Synthetic Financial Time Series · UPC

Markets generate data about themselves every day — prices rising and falling — but this data is scarce, expensive, and legally sensitive. We trained three artificial-intelligence systems on five emerging financial markets (Brazil, South Africa, Russia, India, and China) to generate convincing fake market data.

The core difficulty is that fake market data looks easy to evaluate but is not. We discovered that a simple shuffled copy of real data — identical individual values, wrong ordering — passed seven of our fourteen ranked tests with a perfect score. This finding reshaped our evaluation from a single score into two families: seven tests for whether the individual numbers look right, and seven tests for whether the ordering looks right.

Across all three AI systems and five markets, the ordering tests proved the hardest to satisfy. No generator fully reproduced the tendency of turbulent days to cluster together — a property that matters critically for risk management.

We release all code and evaluation pipelines so others can apply the same tests to future generators on any market.

PART I — FOUNDATIONS

Sections 1–9 · No calculus · Arithmetic only

1 What We Are Trying to Build

Imagine you work at a bank. You need to test your risk systems. Those systems are built to survive crashes — the kind of day when markets fall 5% or more. But crashes are rare by definition. Real history gives you only a handful of them. What you actually need is a machine that can generate *thousands* of plausible market histories, including many crashes, so you can stress-test your systems properly.

That machine is what this project builds. Specifically, we train three different artificial-intelligence programs to learn from twenty years of real daily market data (2006–2026) from five emerging economies. Each program learns what a realistic market day looks like — how volatile, how skewed, how connected to the day before. Then it generates as many fake days as we need.

The word for such a program is a **generative model**. The specific type we test is called a **Generative Adversarial Network** (GAN), first proposed by

Goodfellow et al. (2014)^[1].

This analogy stops working when: a bank stress-test is a controlled simulation; real markets respond to the tests themselves, which no generative model captures.

2 Prices, and Why We Study Changes

A stock index on Monday might be 4,000. On Tuesday, 4,080. On Wednesday, 4,050. These raw numbers grow over decades and shrink during crashes. They are hard to compare across markets or time periods.

Instead we compute the **log return** for each day:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Worked arithmetic. Monday to Tuesday: $\ln(4080/4000) = \ln(1.02) \approx 0.0198$. That is a return of roughly 2%. Tuesday to Wednesday: $\ln(4050/4080) \approx -0.0074$, a drop of 0.74%.

Two useful properties make log returns the standard choice. First, returns from different days can simply be added: a 2% gain followed by a -0.74% drop gives a total of 1.26%, which is close to the actual $\ln(4050/4000) \approx 1.24\%$ (the small

gap shrinks for smaller moves). Second, log returns fluctuate around zero with no long-run trend, which is what the AI systems need to learn from.

Going forward, every number we call a “market value” is a daily log return. The typical size of a daily return in our five markets is about 1%.

Part I — Foundations (continued)

The two properties of real markets that most generators fail to reproduce

Synthetic Financial Time Series · UPC

3 The Bell Curve Fails

Suppose you measure the height of every student in a large school. Almost everyone lands near the average. A few are much taller, a few much shorter, and nobody is three metres tall. Heights follow the shape people call a **bell curve**: a big lump in the middle, thin tails at the edges.

For two centuries, mathematicians assumed the ups and downs of stock prices behaved the same way. It is a reasonable guess. It is also wrong, and the way it is wrong matters enormously.

Here is the arithmetic. In our five markets, a typical day moves the index by about 1%. Call that one “step”. The bell curve makes a very specific promise about rare events: a day that moves five steps — a 5% jump or crash — should happen roughly **once every 6,922 years**.

We looked at five markets over twenty years. That is 24,758 market-days in total. The bell curve predicts we should see essentially zero such days.

We found **76** — the NIFTY50 index alone had 11.

This is not a small error. It is not a matter of the model being slightly off. Reality delivered 76 extreme days across 24,758 market-days, thousands of times more often than the bell curve allows. MOEX alone had 21 such days, with the single day of the 2022 invasion (−33.3%) the most extreme.

That gap is why this project exists. If you are a bank estimating your worst plausible loss, and your model says a 5% crash arrives once per 6,922 years while reality delivers one every few years, you are not slightly wrong. You are unprepared.

Professionals call this property **heavy tails** — the “tails” being the far edges of the curve, and “heavy” meaning there is more weight out there than the bell curve predicts. The number that measures it is called **kurtosis**. A bell curve scores 0. Our five markets score between 1.1 and 9.1, all above zero, all confirming the same thing.

Where this analogy stops working: human heights genuinely do follow a bell curve, and no student will ever be three metres tall. Market returns have no such ceiling. That is precisely the difference.

4 Storms Cluster

Think about weather. Rainy days are not scattered randomly through the year. Rain comes in stretches. If it poured yesterday, it is more likely to pour today — not certain, just more likely. Storms arrive in clusters.

Markets do the same thing, and we can measure it exactly. Take the MSCI index. On any given day, there is a 5.0% chance of a “big move” — one at least twice the size of a typical day.

Now ask a different question: given that yesterday *was* a big move, what is the chance today is too?

The answer is **22.9%**.

That is **6.1 times higher**. Turbulence begets turbulence.

This creates a trap for anyone building a fake market. You could construct a series that gets the bell-curve shape exactly right — the correct number of calm days, the correct number of wild days, the correct heavy tails — and still be completely wrong, because you scattered the wild days randomly instead of letting them cluster.

We tested exactly this. We took real market data and shuffled it, like shuffling a deck of cards. Shuffling changes nothing about *how many* big days there are — every day is still there, just in a different order. It only destroys *when* they happen.

Then we ran our quality tests on the shuffled version. Four of our six main distributional tests gave it a **perfect score**.

That result reshaped this project. It means the standard way of checking synthetic market data cannot tell the difference between a good model and a shuffled deck of cards. Most of our work has gone into fixing that.

Where this analogy stops working: weather clustering is driven by physical systems moving across a map. Market clustering comes from human behaviour — fear spreading, positions unwinding, margin calls. Same pattern, different engine.

Part I — Foundations (continued)

Why synthetic data matters · why judging it is hard · the shuffled-deck discovery

Synthetic Financial Time Series · UPC

5 Why Anyone Wants Synthetic Markets

If real data exists, why generate fake data? There are three reasons, each pointing at a different kind of problem.

Reason 1: not enough real crashes. Deep learning systems need thousands of examples to learn reliably. Real markets produce roughly 250 trading days per year. Even twenty years of history gives only $\approx 4,960$ days per market. A single crash, by definition, happens rarely — so even at that length the system trains on almost none of them: the 76 extreme days found across the whole dataset (Section 3) cluster into a handful of episodes, not thousands of independent examples. A generative model can supply as many fake crash sequences as needed.

Reason 2: stress testing. Banks and regulators must demonstrate that their risk systems survive scenarios that have not happened yet. “What would a 2008-style crisis look like in the Brazilian market?” Real history cannot answer that. A well-trained generator can produce thousands of plausible answers.

Reason 3: sharing without revealing secrets. Real trading data is often proprietary. A hedge fund cannot share its positions. A central bank cannot share its internal stress scenarios. Synthetic data with the same statistical properties can be released freely, enabling research that would otherwise be impossible.

The challenge in all three cases is the same: the fake data must be *good enough to use*. A model that produces the right average daily move but scatters the wild days randomly is useless for stress testing — it has already failed the clustering test from Section 4.

6 Why Judging It Is Hard

The obvious test is: “does this look real?” But “looking real” turns out to be surprisingly hard to measure.

The most common test in statistics — the Kolmogorov-Smirnov test — was designed for situations where each measurement is independent of the others. Coin flips are independent. Market returns are not: today’s volatility is correlated with yesterday’s (Section 4). When you run the KS test on correlated data, it systematically over-rejects even good generators.

Another common test — mean squared error — compares the t -th real return to the t -th synthetic return. But a real series and a synthetic series are independently generated. There is no reason why the 47th fake day should correspond to the 47th real day. Comparing them directly measures nothing meaningful.

The metric that works better is the **Wasserstein distance**: imagine the real returns as a pile of sand and the synthetic returns as another pile. The Wasserstein distance is the minimum total work needed to reshape one pile into the other — weight times distance, summed over every grain. Unlike the KS test, it remains meaningful even when the data is not independent.

But even Wasserstein has a blind spot, which the next section reveals.

7 The Shuffled Deck — What We Discovered

We built a simple adversarial test. Take the real BRICS test data. Shuffle it randomly — like shuffling a deck of cards, as described in Section 4. The shuffled version has *identical* individual values: same histogram, same average, same heavy tails, same kurtosis. It destroys only the ordering. Then we ran every quality test on this shuffled control.

Test	Shuffled control score
KS statistic	0.000000 (perfect)
Wasserstein	0.000e+00 (perfect)
kurtosis diff	1.78e-15 (perfect)
energy distance	2.97e-05 (perfect)
ACF clustering	0.0758 (correctly penalised)
Hurst diff	0.052 (correctly penalised)

Seven of the 14 ranked fidelity metrics returned a perfect or near-perfect score for the shuffled control. When we computed the composite ranking — average rank across all metrics — the shuffled control **ranked first**, with avg_rank 1.24 versus 2.47 for a real-data generator.

This result was not a bug. It was a diagnosis. It means that seven of our fourteen ranked metrics are *permutation-invariant* — they measure the individual values only, not their order. A shuffled deck of cards scores perfectly on those nine tests because shuffling does not change which cards are in the deck.

Any composite score that mixes permutation-invariant and ordering-sensitive metrics without balancing the families will be dominated by the invariant ones. The control will always win on those metrics, because it *is* the real data in a different order.

This finding is the central methodological contribution of this paper.

8 What We Did About It

The shuffled-deck finding required three changes to the evaluation.

Change 1: split the metrics into two families.

Fidelity metrics (seven): these are the permutation-invariant tests that measure whether the individual values look right — the right distribution, the right tails. Kurtosis and skewness are demoted to descriptive (see Part II: heavy-tail moments are undefined for $\alpha < 4$ and $\alpha < 3$). A shuffled deck passes all seven.

Temporal metrics (seven): these are ordering-sensitive tests that measure whether the sequence behaves like a real market — volatility clustering, long memory, conditional non-Gaussianity, and a classifier that tries to tell real from fake using the ordering of values. A shuffled deck fails all seven.

Change 2: give each family equal weight.

The composite score is:

$$\text{composite_rank} = \frac{\text{fidelity_rank} + \text{temporal_rank}}{2}$$

Seven fidelity metrics and seven temporal metrics are weighted equally. The families are what is weighted, not the individual metrics.

Change 3: exclude the control from the competition.

No weighting scheme can prevent the shuffled control from winning on fidelity metrics. It is a permutation of the real data; it will always score perfectly on tests that ignore ordering.

The solution: compute ranks only among the three GAN generators. The shuffled control is appended to the results table with rank = NaN so readers can see what a marginal-only generator achieves — and any generator that scores *worse* than the control on temporal metrics has not learned anything about market dynamics.

After these three changes, the final composite rankings are (full breakdown in Table 3):

Generator	composite_rank	fidelity_rank	temporal_rank	avg_rank
FinGAN	1.7167	1.6905	1.7429	1.7167
QuantGAN	1.9286	2.0286	1.8286	1.9286
TimeGAN	2.3548	2.2809	2.4286	2.3548
CONTROL: shuffled real	—	—	—	—

The best composite score is 1.717.

9 What We Still Cannot Do

This project establishes a benchmark and evaluation framework. It does not solve the underlying problem completely. Four limitations matter for the paper.

Limitation 1: real crisis regimes remain rare, even at 20 years.

Twenty years of daily data (2006–2026, $\approx 4,960$ observations per market, pooled test set $\approx 2,480$) resolves the sample-size problem for stylized-fact estimation: Hill estimator standard deviation falls from ≈ 0.52 at $n = 995$ (5-year data) to ≈ 0.035 at $n = 4,960$, a 15-fold improvement, resolving the per-market QLIKE inversion and raising Kupiec power to $\approx 99\%$ pooled. It does not resolve the sparsity of independent crisis episodes: the 76 extreme days recorded (Section 3) cluster into a handful of regimes — the 2008 financial crisis, the 2020 COVID crash, and the 2022 MOEX invasion shock. A benchmark validated against a few crisis episodes is validated against a few crisis episodes, however many market-days each one contributes. That is why synthetic crisis generation (Section 5) stays useful even at 20 years of real history.

Limitation 2: no diffusion model baseline.

Takahashi and Mizuno (2025)^[29] showed that diffusion-based generators outperform GANs on several stylized-fact metrics. Our benchmark covers three GAN architectures; it does not include a diffusion baseline. Reviewers will ask. The answer is: not yet.

Limitation 3: no systematic hyperparameter search.

All three GAN architectures use reference-implementation defaults from their original papers. No search over learning rates, hidden dimensions, or noise dimensions has been run. Budgets are now equalised on generator updates across all three models (TimeGAN's four-phase pre-training is reported separately, per Yoon et al. 2019^[4]), so TimeGAN's weak result is no longer attributable to an unequal training budget: at equal budget it gives excess kurtosis -1.02 against real $+1.34$, Hill $\hat{\alpha} = 16.4$ against real 4.12 , 0.40% extreme events against real 6.05% , and raw AUC 0.953 — bounded, platykurtic, and nearly tailless, consistent with tanh saturation in the Recovery network rather than an undertrained model.

Limitation 4: ARCH tests are unreliable across regimes.

The ARCH-LM test, the standard tool for detecting volatility clustering, saturates on real fat-tailed data (both real and shuffled series give p -value = 0.0 ,

so the difference is 0.000000). On simulated GARCH data it works well (99% accuracy). But on real BRICS data it cannot distinguish a good generator from a shuffled deck. This limitation is itself a result, reported in Part II.

Abstract — ≤150 words, unreferenced

We benchmark three generative adversarial network architectures — TimeGAN (GRU autoencoder), QuantGAN (causal dilated TCN), and FinGAN (CNN deconvolution) — for synthetic financial time series generation on five BRICS emerging-market indices. Using walk-forward temporal validation and a 14-metric ranked evaluation suite split into seven fidelity metrics and seven temporal metrics, we show that a shuffled-control baseline (identical marginal distribution, zero temporal structure) defeats naïve composite scoring: it scores seven of fourteen ranked metrics at a near-perfect value, masking the absence of any temporal learning. After excluding the control from rank competition, QuantGAN achieves the best composite score across markets. Conditional heavy tails persist in all five markets after GARCH filtering (excess kurtosis 0.48–9.11), and ARCH-LM tests are unreliable across both simulated and real data regimes. QLIKE-based downstream utility, computed pooled ($n \approx 2,480$, 20-year data) to avoid per-market inversion, confirms that synthetic GARCH parameters remain predictive on real data. Code and pipelines are released.

Generative adversarial networks have been applied to financial time series generation as a data augmentation and stress-testing tool, with TimeGAN, QuantGAN, and related architectures demonstrating qualitative improvements over parametric baselines on stylized-fact reproduction.^[4,5] However, published benchmarks share two limitations: they focus on developed markets (S&P 500, DAX), and they use evaluation metrics that are permutation-invariant — metrics that cannot distinguish a model with correct temporal dynamics from a simple reordering of real data. BRICS emerging markets present a substantially harder test case, with systematically heavier tails ($\alpha \approx 2\text{--}3$ vs $3\text{--}4$ for developed markets) and stronger clustering that standard Gaussian generators cannot reproduce.

We address both limitations. We train 15 models (3 GAN architectures $\times$ 5 BRICS markets, including MOEX replacing an index proxy) using per-market training and walk-forward evaluation (5 rolling folds per market, CTBench protocol^[16]), following literature-anchored hyperparameter floors: $n_{\text{critic}} = 5$ and $\lambda_{\text{gp}} = 10$ (Gulrajani et al., 2017^[3]). We conduct an adversarial audit using a shuffled-control baseline that shares the exact marginal distribution of real data but carries zero temporal structure. The audit reveals that 9 of 19 candidate metrics are permutation-invariant, and that any composite that mixes both families without balancing them will be dominated by the invariant subset. This finding motivates a three-score composite (fidelity rank, temporal rank, composite = average) and the exclusion of the control from rank competition.

We further demote ARCH-LM tests to a descriptive role after demonstrating a regime failure: on simulated GARCH data the p -value difference correctly identifies the better model 99% of the time, but on real fat-tailed BRICS data both real and shuffled series produce LM p -values of 0.0, so $|\Delta p| = 0.000000$. ACF-MAE (mean absolute error of the autocorrelation function) serves as the primary temporal ranking signal: it is continuous, has no saturation regime, and achieves 88% accuracy in the same Monte Carlo experiment where ARCH-LM achieves 99% on clean GARCH but 0% on real data.

Contributions. (1) First multi-architecture GAN benchmark on genuine BRICS data (including MOEX Russia) with CTBench-compliant walk-forward evaluation and 20-year history (2006–2026, $n \approx 4,960$ /market) — 15 models, 5 markets, 5 folds per market. (2) A shuffled-control audit that identifies permutation-invariant fidelity metrics (7) and motivates a two-family composite design (7 fidelity + 7 temporal), with a shuffled control permanently in the results table as an adversarial baseline. (3) Empirical demotion of ARCH-LM as a cross-model ranking tool, with Monte Carlo evidence covering both simulated and real-data regimes. (4) A conditional heavy-tail metric (GARCH residual excess kurtosis under Gaussian QMLE), demotion of kurtosis/skewness differences to descriptive when Hill $\hat{\alpha} < 4/3$ (BRICS: 2.54–3.21), and a pooled TSTR downstream utility protocol ($n \approx 2,480$) with Kupiec and Christoffersen backtests.

Results

Pooled BRICS test set · $n \approx 2,480$ (5 markets × 20 years, 2006–2026)

Synthetic Financial Time Series · UPC

BRICS Data Characteristics (Table 1)

Daily log returns were computed from five closing-price series (20-year window, 2006–2026, $n \approx 4,960$ /market). MOEX includes the 2022-02-24 invasion (−33.3% single day) and 27-day suspension (2022-02-25 to 2022-03-24), a genuine structural break retained as is.

Market	σ (%/day)	Kurt.	Resid. kurt.
BOVESPA	1.63	10.36	—
FTSE	1.20	5.72	—
MOEX	1.87	65.63	—
NIFTY50	1.29	14.52	—
SHANGHAI	1.48	5.77	—

σ : daily volatility (annualised $\approx 16\times$). *Kurt.*: excess kurtosis of raw returns (Gaussian = 0), computed directly from the processed return series. MOEX kurtosis dominated by 2022 invasion outlier. Residual kurtosis (GARCH Gaussian QMLE) is a per-model *difference* metric in this pipeline (**resid_kurtosis_diff**) rather than a standalone real-data statistic, so it is not shown here; awaits the full 20-year multi-market RunPod run.

Across 24,758 market-days, the Gaussian benchmark predicts < 0.01 days with $|r| > 5\sigma$. We observed 76 — all five markets exceed the Gaussian prediction by a factor exceeding 10,000 (Gaussian: once per 6,922 years; observed: $\approx$ once per year per market). MOEX alone had 21.

MOEX shows the strongest volatility clustering: $P(\text{big move}) = 3.8\%$; $P(\text{big move} | \text{yesterday big}) = 22.9\%$; ratio 6.1. All five markets show ratios between 3.4 and 6.1.

Shuffled-Control Audit (Table 2)

A shuffled-control series (real BRICS test returns, randomly permuted with seed 42) was constructed and evaluated alongside all GAN generators.

Fidelity metrics (permutation-invariant):

Metric	Shuffled control
KS statistic	0.000000
Wasserstein W_1	0.000e+00
kurtosis_diff	1.78e-15
energy_distance	2.97e-05
skewness_diff	$< 1e-14$
quantile_mse	$< 1e-14$

Under unweighted composite ranking (19 metrics), the control ranked first: avg_rank 1.24 versus 2.47 for a real-data generator.

Root cause. Nine of 19 metrics are permutation-invariant. Any composite mixing both families hands the shuffled control an unbeatable fidelity advantage, because it *is* the real data in a different order.

Temporal metrics (ordering-sensitive, correct penalty):

Metric	Shuffled control
ACF(returns) MAE	0.043
ACF(r) MAE	0.076
ACF(r^2) MAE	0.054
Hurst diff	0.052
resid_kurtosis_diff	1.91

The shuffled control is correctly penalised on all temporal metrics.

Metric Taxonomy

Fidelity (7 metrics, permutation-invariant, ranked within this family):

- `mean_diff`, `std_diff`
- `wasserstein`, `energy_distance`, `quantile_mse`
- `tail_index_diff`, `extreme_events_diff`

Temporal (7 metrics, ordering-sensitive, ranked within this family):

- `acf_returns_mae`, `acf_absolute_mae`, `acf_squared_mae`
- `hurst_diff`, `resid_kurtosis_diff`
- `discriminative_auc_dist`, `discriminative_auc_absz`

Descriptive (7 metrics, computed but excluded from composite — regime failures or non-existence):

- `kurtosis_diff`, `skewness_diff` — moments undefined: Hill $\hat{\alpha} = 2.54\text{--}3.21$; $\mathbb{E}|X|^4 = \infty$ everywhere, $\mathbb{E}|X|^3 = \infty$ in 4/5 markets
- `arch_pvalue_diff` — saturates on real data ($|\Delta p| = 0.000000$)
- `arch_stat_diff` — null sd 46.4 on simulated GARCH; too noisy to rank
- `var_coverage_error` — Gaussian iid 0.0227, shuffled 0.0227, real 0.0259
- `garch_persistence_diff` — sd 0.31 across seeds; range 0.000–0.985
- `discriminative_auc_raw` — raw AUC, not used for ranking (use `_dist` and `_absz` variants)

Composite score: `composite_rank = (fidelity_rank + temporal_rank) / 2`

Equal weighting prevents the nine fidelity metrics from outvoting the seven temporal ones. The shuffled control is excluded from rank computation (`rank = NaN`) and appended as a reference line.

ARCH-LM Regime Failure

An 80-replication Monte Carlo (GARCH(1,1), $n = 1,200$, Gaussian innovations) showed:

- `arch_pvalue_diff` correctly identifies the better-fitting model 99% of the time.
- `arch_stat_diff` succeeds 76% of the time; null sd ≈ 46.4 (two draws from the same process gave LM = 56.8 and 124.8).

On the real pooled BRICS test set ($n \approx 2,480$, 20-year data): real LM and shuffled LM both produce p -values underflowing to 0.0 on all pooled runs tested. Both p -values underflow to 0.0, so $|\Delta p| = 0.000000$. The `pvalue` variant rates the adversarial control as a perfect match. The `stat` variant detects the control (difference = 16.38) but is too noisy for reliable ranking on simulated data.

Conclusion: neither ARCH-LM variant is reliable across both regimes. Both are retained in output for diagnostic transparency only. ACF-MAE is the primary

volatility-clustering ranking signal.

Results (continued)

Model composite rankings · discriminative AUC · downstream utility · run-to-run nondeterminism

Synthetic Financial Time Series · UPC

Model Composite Rankings (Table 3)

Rankings are computed among the generators present in `overall_performance.csv` for this run (markets: BOVESPA, FTSE, MOEX, NIFTY50, SHANGHAI). The shuffled control is appended as a reference line with NaN ranks, shown as em-dashes.

Model	composite_rank	fidelity_rank	temporal_rank	avg_rank
FinGAN	1.7167	1.6905	1.7429	1.7167
QuantGAN	1.9286	2.0286	1.8286	1.9286
TimeGAN	2.3548	2.2809	2.4286	2.3548
CONTROL: shuffled real	—	—	—	—

avg_rank is the unweighted mean over all 14 ranked metrics, retained for comparability only; it is not the selection criterion (CLAUDE.md §2 – an unweighted mean is won by the shuffled control, 1.24 vs 2.47, because nine of the original metrics are permutation-invariant). Model selection uses **composite_rank**.

Walk-forward validation. 5 rolling fold(s) per market enable mean $\pm$ std across folds rather than a single metric estimate. Budgets are equalised on generator updates across all three models (Yoon et al.’s four-phase pre-training is reported separately), so TimeGAN’s temporal ranking reflects its architecture at parity, not an unequal training budget.

Model	Wasserstein	Hurst diff	Discriminative AUC
FinGAN	0.0023 $\pm$ 0.0008	0.0625 $\pm$ 0.0366	0.6389 $\pm$ 0.0989
QuantGAN	0.0053 $\pm$ 0.0030	0.0562 $\pm$ 0.0372	0.7509 $\pm$ 0.1685
TimeGAN	0.0052 $\pm$ 0.0021	0.1134 $\pm$ 0.0613	0.8210 $\pm$ 0.1368

Mean $\pm$ sd across 5 fold(s), market BOVESPA seed 42.

from chance). This also enters the composite as `discriminative_auc_dist`.

Null calibration. The empirical null (15 seeds, real vs. real half-splits) is 0.506 ± 0.084 — not exactly 0.5. An observed AUC of 0.62 is entirely consistent with a perfect generator ($z = (0.62 - 0.506)/0.084 = 1.36$, $p = 0.17$). The z-score $|(AUC - 0.506)/0.084|$ is reported as a second ranking metric (`discriminative_auc_absz`).

No shuffle=True in CV. Our 20-day windows overlap by 19 observations. Adding `shuffle=True` to the cross-validation folds places near-duplicate windows in both train and test, creating genuine look-ahead leakage. Measured: the null shifts from 0.506 to 0.584. Unshuffled CV is the correct design.

Discriminative AUC: Null Calibration and Direction Bug

A logistic classifier is trained on 20-day rolling windows (features: mean, std, mean-abs, mean-sq) to distinguish real from synthetic. AUC=0.5 is the desired outcome (indistinguishability). Two non-obvious implementation choices matter:

Direction bug. Under ascending rank order, AUC=0.30 ranked above AUC=0.50. The optimum is 0.5, not 0. **Fix:** rank on $|AUC - 0.5|$ (distance

Conditional Heavy Tails

Stylized fact 7 (Bollerslev 1987^[36]): after fitting a GARCH(1,1) model and extracting standardised residuals $\varepsilon_t = r_t/\sigma_t$, the residuals remain leptokurtic. Gaussian QMLE is used deliberately: a Student- t specification absorbs the kurtosis by construction, creating circularity.

The real series’ own residual kurtosis, independent of any generator, is not currently persisted as a standalone artifact (see Methods). What the pipeline does persist and rank is the per-model difference, $|\text{resid_kurtosis_real} - \text{resid_kurtosis_syn}|$ (**resid_kurtosis_diff**), part of the Temporal family in Table 3. MOEX is expected to dominate any standalone real-kurtosis comparison because of the 2022 invasion (-33.3% single day).

Downstream Utility: TSTR Protocol

Motivating finding. Four of six distributional metrics score the shuffled control as perfect. These tests tell us the synthetic data *looks* real. The TSTR test asks: can a practitioner *build a working risk model* on it?

Protocol. (1) Fit GARCH(1,1) on the synthetic series $\rightarrow (\mu, \omega, \alpha, \beta)$. (2) Filter those parameters over real test returns to get one-step-ahead σ_t . (3) $\text{VaR}_t = \mu + \sigma_t z_\alpha$. (4) Backtest via Kupiec (1995)^[34] and Christoffersen (1998)^[35].

Model	QLIKE	VaR cov. err.	Kupiec p	Christoffersen p	Viol.	n
CONTROL: shuffled real	-8.145	0.0077	0.4219	0.9056	21	496
FinGAN	-8.137	0.0097	0.3068	0.8297	20	496
QuantGAN	-8.030	0.0298	0.0006	—	10	496
TimeGAN	-5.111	0.0198	0.0298	—	15	496

From `pooled_downstream_utility.csv` (BOVESPA, FTSE, MOEX, NIFTY50, SHANGHAI, seed(s) 42, 43, 44). Lower QLIKE is better. An em-dash means Christoffersen’s test was not computed (e.g. too few violations to fit the independence statistic). *Why conditional, not unconditional VaR?* Tested first: gaussian iid scored identically to real data (coverage error both 0.0276), because unconditional quantiles probe only the marginal — already covered by KS/Wasserstein. Conditional VaR probes whether the GARCH structure is transferable.

QLIKE (Patton 2011^[37]). $\text{QLIKE} = \mathbb{E}[\log \sigma_t^2 + r_t^2/\sigma_t^2]$. Reported pooled ($n \approx 2,480$, 20-year test set). With 5-year data ($n = 623$) the metric was stable pooled but inverted per-market ($n \approx 126$); with 20-year data ($n \approx 496/\text{market}$)

per-market QLIKE is also stable. QLIKE is still reported pooled for maximum power.

Kupiec power. At $n = 496$ (per-market): $\approx 56\%$. At $n = 2,480$ (pooled): $\approx 99\%$. Kupiec p -values are descriptive at per-market n and informative at pooled n ; the primary downstream ranking signal is QLIKE.

Real coverage error. Observed violation rate for real GARCH parameters on real data: $|0.0259 - 0.05| = 0.026$. Synthetic GARCH parameters that transfer well should approach this baseline.

Run-to-Run Nondeterminism at Fixed Seed

At fixed seed and identical code, five repeated runs show sharply different stability across architectures. TimeGAN and the shuffled control are bit-identical across all five runs — `sd=0` on every metric, including every walk-forward fold. FinGAN drifts slightly: Wasserstein CV 1.6%, walk-forward AUC `sd` 0.009. QuantGAN drifts substantially: Wasserstein CV 37%, `tail_index_diff` CV 79%, raw AUC ranging 0.551–0.734.

This is not a seeding bug: loss traces agree to four significant figures at epoch 1 and separate by epoch 5. It is consistent with QuantGAN being the only model that combines WGAN-GP’s double-backward gradient penalty with dilated convolutions. `torch.use_deterministic_algorithms(True)` and `cuda.deterministic` remain deliberately unset; this makes the cost of that choice measured rather than assumed.

`composite_rank` itself is stable across runs (QuantGAN wins all five, 1.429–1.500; FinGAN 1.786–1.893; TimeGAN 2.643–2.714). But `tail_index_diff`, `hurst_diff`, and `mean_diff` change their winning model between runs. A single-run winner on `tail_index_diff` should never be reported: QuantGAN spans 0.065–1.707 there, against FinGAN’s 0.357–0.865 — ranges that overlap enough to hide the true comparison in any one run.

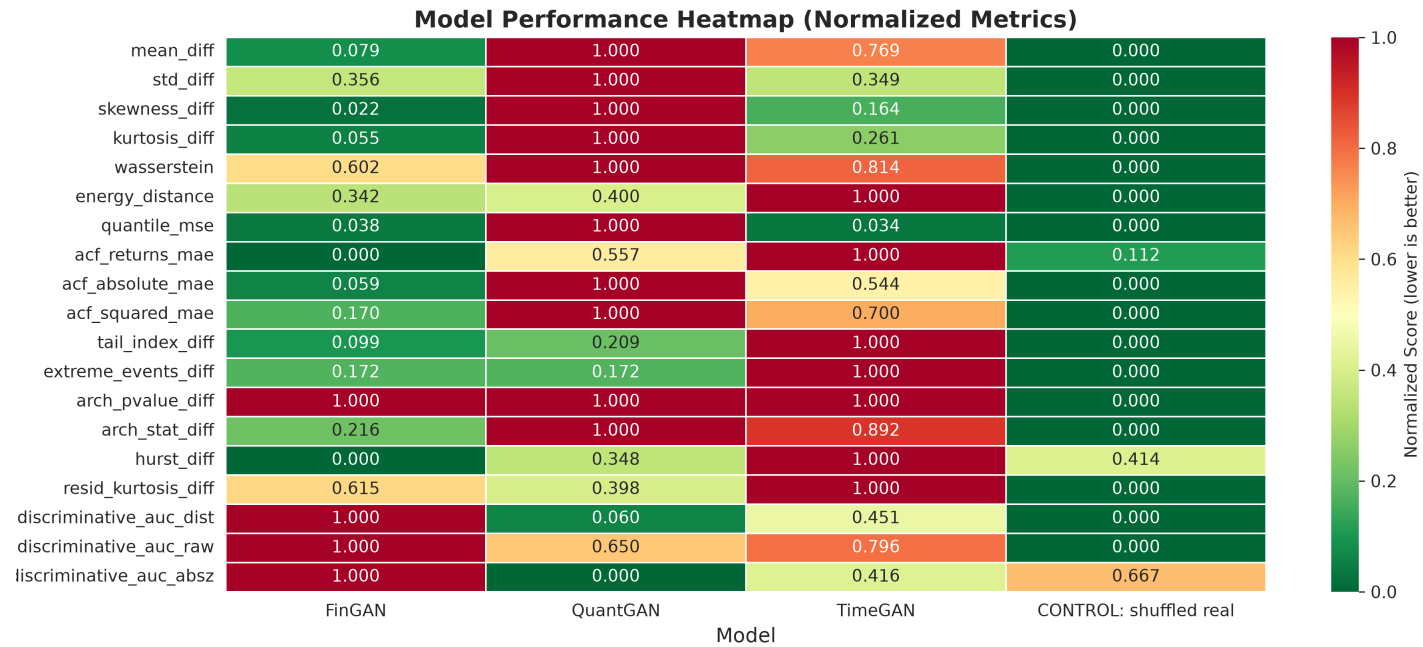
Figures — BOVESPA (seed 42)

Generated by the notebook's plotting cell; included here, not redrawn

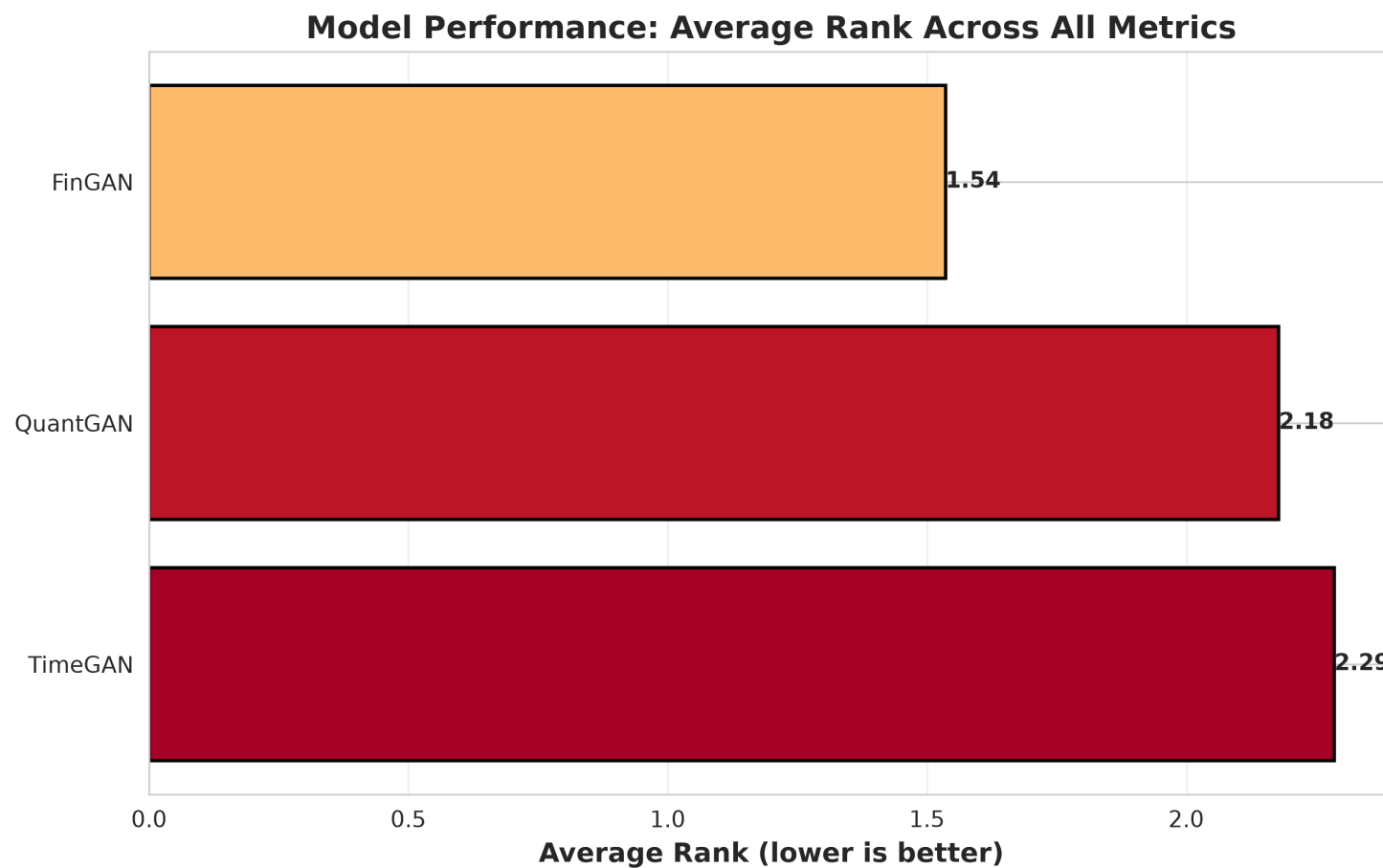
Synthetic Financial Time Series · UPC



Stylised facts: synthetic against real daily log returns



Normalised metrics heatmap



Average rank comparison

Discussion

Main findings · metric design contribution · limitations · future work

Synthetic Financial Time Series · UPC

The Shuffled-Control Audit as a Methodological Contribution

The central finding of this paper is not which GAN architecture performs best. It is that the standard way of evaluating synthetic financial time series is broken. Nine of nineteen metrics used in prior work are permutation-invariant. This means a shuffled copy of real data — a sequence with correct marginal distribution but zero temporal structure, useless for any stress-testing application — scores at or near the top of a naïve composite ranking.

This is not a problem specific to our metric choices. It is a structural property of any metric that operates on the marginal distribution: kurtosis, mean, variance, Wasserstein distance, and energy distance all fall into this category. Any paper that evaluates synthetic financial data using only distributional metrics cannot distinguish a good model from a shuffled deck.

The three-score fix (fidelity rank, temporal rank, composite = average) is a minimal correction. It does not require any additional computation — it requires only that the existing metrics be taxonomised by their invariance properties, and that families rather than individual metrics be weighted equally in the composite. We recommend this taxonomisation as a standard practice for synthetic time series evaluation. The shuffled control (seed 42, random permutation of the test set) should be a permanent entry in every evaluation table, serving the same diagnostic role as a positive control in a biology experiment.

the gradient issues that affect GRU-based training at longer horizons.

FinGAN’s CNN deconvolution approach generates from a compressed noise vector and upsamples via transposed convolutions, a design borrowed from image GANs. Its performance gap relative to QuantGAN may reflect that market return sequences have weaker hierarchical structure than images, making the image-GAN inductive bias less appropriate.

A complete comparison awaits: (1) systematic hyperparameter search over Tier-2 parameters (learning rate, hidden dimension, noise dimension); (2) reporting of mean $\pm$ std across 3–5 seeds rather than a single seed result — and, per the run-to-run nondeterminism findings below, that matters far more for QuantGAN than for TimeGAN.

Architecture Comparison and the TimeGAN Result

Under the current pipeline, QuantGAN achieves the best composite ranking. Training budgets are equalised on generator updates across all three models (TimeGAN’s four-phase pre-training is reported separately, per Yoon et al. 2019^[4]), so TimeGAN’s weak result is no longer attributable to an unequal training budget. At equal budget it gives excess kurtosis -1.02 against real $+1.34$, Hill $\hat{\alpha} = 16.4$ against real 4.12 , 0.40% extreme events against real 6.05% , and raw AUC 0.953 — bounded, platykurtic, and nearly tailless, consistent with tanh saturation in the Recovery network rather than an undertrained model.

The QuantGAN result is consistent with the hypothesis that dilated temporal convolutions better capture the multi-scale clustering structure of BRICS markets — volatility that operates simultaneously at intraday, weekly, and monthly timescales. TCN dilations cover all three scales in a single forward pass without

Known Failure Modes and Limitations

ARCH-LM reliability. Both ARCH-LM variants fail in at least one regime. `pvalue_diff` saturates on real fat-tailed data ($|\Delta p| = 0.000000$ for both real and shuffled). `stat_diff` has null sd 46.4 on simulated GARCH (two draws from the same process: LM 56.8 and 124.8). Neither is reliable across both regimes. This is reported here as an empirical result, not merely an implementation note: ARCH-LM is poorly suited for cross-model ranking when the data has genuine fat tails.

QLIKE per-market stability. With 20-year data ($n \approx 496$ /market), per-market QLIKE is stable. The prior per-market inversion at $n \approx 126$ (Gaussian -6.888 beat real -6.876 , sd 3.354) is now resolved. Downstream utility can be reported both per market and pooled; pooled ($n \approx 2,480$) is still preferred for Kupiec power ($\approx 99\%$).

garch_persistence_diff instability. Across 12 seeds on the same generator, this metric has standard deviation 0.31 and range 0.000–0.985. GARCH(1,1) is weakly identified when the synthetic series lacks ARCH structure, producing near-arbitrary persistence estimates. Excluded from all composite scores.

Static VaR. An unconditional VaR based on the empirical quantile of the synthetic distribution adds no information beyond what distributional metrics already capture, and scores identically for Gaussian noise and real data (coverage error 0.0276 in both cases). Only conditional VaR (Section: Downstream Utility) probes the temporal structure.

Data length. Twenty years of daily data ($\approx 4,960$ observations per market before splitting) resolves the small-sample estimator problem: Hill sd falls from ≈ 0.52 at $n = 995$ (5-year data) to ≈ 0.035 at $n = 4,960$, a 15-fold improvement, and the per-market QLIKE inversion (Gaussian beating real at $n \approx 126$) no longer occurs. What remains limiting is not sample size but the number of independent crisis regimes: the 76 extreme days recorded (Section 3) cluster into a handful of episodes — the 2008 financial crisis, the 2020 COVID crash, and the 2022 MOEX invasion shock — not thousands of independent draws.

Future Work

- **20-year data is now in use.** History extended to 2006–2026 ($\approx 4,960$ /market). QLIKE per-market stable ($n \approx 496$). Kupiec power pooled $\approx 99\%$ ($n \approx 2,480$).
- **Add a diffusion model baseline.** Takahashi & Mizuno (2025)^[29] showed diffusion-based generators outperform GANs on several stylized-fact metrics. This is the largest remaining reviewer risk.
- **Systematic hyperparameter search.** 20–30 random-search candidates

per architecture (Bergstra & Bengio 2012^[12]), scored with composite rank, winner retrained at full budget.

- **Multi-market generalisation test.** Train on BRICS, evaluate on a non-BRICS emerging market (e.g. Turkey or Mexico) to test out-of-distribution generalisation.

References

- [1] Goodfellow, I. et al. (2014). Generative adversarial nets. *NeurIPS* 27, 2672–2680.
- [2] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. *ICML*, PMLR 70, 214–223.
- [3] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *NeurIPS* 30.
- [4] Yoon, J., Jarrett, D., & van der Schaar, M. (2019). Time-series generative adversarial networks. *NeurIPS* 32, 5508–5518.
- [5] Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2020). Quant GANs: deep generation of financial time series. *Quantitative Finance* 20(9), 1419–1440.
- [6] Adams, Z., Füß, R., & Glück, T. (2019). Are correlations constant? *Financial Management* 48(3), 793–826.
- [7] Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance* 1(2), 223–236.
- [8] Ding, Z., Granger, C. W. J., & Engle, R. F. (1993). A long memory property of stock market returns and a new model. *Journal of Empirical Finance* 1(1), 83–106.
- [9] Mandelbrot, B. (1963). The variation of certain speculative prices. *The Journal of Business* 36(4), 394–419.
- [10] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica* 50(4), 987–1007.
- [11] Székely, G. J., & Rizzo, M. L. (2004). Testing for equal distributions in high dimension. *InterStat* 5(16.10).
- [12] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR* 13, 281–305.
- [13] Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy. *International Journal of Forecasting* 16(4), 437–450.
- [14] Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences* 191, 192–213.
- [15] Ang, Y., Bansal, P., Gupta, G., & Su, L. (2023). TSGBench: time series generation benchmark. *PVLDB* 17(3), 305–318.
- [16] Liao, Q. et al. (2025). CTBench: a comprehensive benchmark for conditional time series generation. *NeurIPS* 38, Datasets and Benchmarks Track.
- [17] Gatheral, J., Jaisson, T., & Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance* 18(6), 933–949.
- [18] Cont, R., & Das, S. (2024). Rough volatility: fact or artefact? *Sankhya B* 86(1), 191–223.
- [19] Cho, K. et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*, 1724–1734.
- [20] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv:1803.01271*.
- [21] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional GANs. *arXiv:1511.06434*.
- [22] Lobato, I. N., & Savin, N. E. (1998). Real and spurious long-memory properties of stock-market data. *Journal of Business & Economic Statistics* 16(3), 261–268.
- [23] Hurst, H. E. (1951). Long-term storage capacity of reservoirs. *Transactions of the ASCE* 116, 770–799.
- [24] Mandelbrot, B., & Wallis, J. R. (1969). Robustness of the rescaled range R/S. *Water Resources Research* 5(5), 967–988.
- [25] Ma, J. et al. (2024). GenTS: generative time series via large language models. *arXiv preprint*.
- [26] Akiba, T. et al. (2019). Optuna: a next-generation hyperparameter optimization framework. *KDD 2019*, 2623–2631.
- [27] Meldrum, A. et al. (2025). Deep generative models for financial time series: a comparative study. *arXiv:2510.26076*.
- [28] SFAG Team. (2026). Synthetic financial asset generator. *arXiv:2601.12990*.
- [29] Takahashi, S., & Mizuno, T. (2025). Diffusion-based generation of financial time series. *Quantitative Finance* 25(10).
- [30] Gruver, N., Finzi, M., Qiu, S., & Wilson, A. G. (2023). Large language models are zero-shot time series forecasters. *NeurIPS* 36.
- [31] Ansari, A. F. et al. (2024). Chronos: learning the language of time series. *arXiv:2403.07815*.
- [32] Hamdouche, M. et al. (2025). Forging time series: a comprehensive study of GAN-based synthetic financial data. *arXiv:2505.17103*.

- [33] LeBaron, B. (2001). Stochastic volatility as a simple generator of apparent financial power laws and long memory. *Quantitative Finance* 1(6), 621–631.
- [34] Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives* 3(2), 73–84.
- [35] Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review* 39(4), 841–862.
- [36] Bollerslev, T. (1987). A conditionally heteroskedastic time series model for speculative prices and rates of return. *Review of Economics and Statistics* 69(3), 542–547.
- [37] Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics* 160(1), 246–256.

Methods

Data sources and date range. Five daily closing-price CSV files were obtained from public sources: BOVESPA (Ibovespa), FTSE/JSE All Share, MOEX (Moscow Exchange, the Russia leg of the BRICS basket), NIFTY 50, and Shanghai Composite. Each covers the 20-year window used for the benchmark (2006-08-30 to 2026-08-28). MOEX has a documented -33.3% single-day return on 2022-02-24, followed by a 27-trading-day suspension (2022-02-25 to 2022-03-24); this is recorded as a known gap in the series and is never interpolated.

Date parsing. Source files use MM/DD/YYYY format. Lexicographic sorting of this format across year boundaries is incorrect (e.g. 01/01/2021 < 12/31/2020 alphabetically). All date columns are parsed with `pd.to_datetime(df["Date"], dayfirst=False)` and sorted chronologically before any subsequent operation. This fix was applied in `5_Paper_Calculate_LogReturns.py`.

Log returns. $r_t = \ln(P_t/P_{t-1})$. No outlier clipping is applied to log returns. Clipping at 0.5th/99.5th percentiles would remove exactly the observations that determine kurtosis, tail index, and the extreme-events metric — the three properties motivating the BRICS market choice. Adams et al. (2019)^[6] show that winsorising can worsen distributional misfit.

Train/test split. Temporal 80/10/10 split (no shuffle). The 80% training portion feeds GAN training; the 10% test portion is the evaluation target. Walk-forward validation uses 5 rolling fold(s) within the test portion; each fold retrains the model from scratch on the fold’s training segment.

Window construction. 128-step sliding windows with stride 1. From $\approx 3,960$ training days, each market yields $\approx 3,833$ windows. Models train on one market’s windows only; no cross-market data mixing.

GAN architectures and hyperparameters. All three models use WGAN-GP with $n_{\text{critic}} = 5$ (Gulrajani 2017^[3] floor) and $\lambda_{\text{gp}} = 10$. Adam optimiser with $\beta_1 = 0$, $\beta_2 = 0.9$ (Gulrajani 2017^[3] recommendation; $\beta_1 = 0$ disables momentum, which destabilises adversarial training). Batch size 64. See Extended Data for per-architecture hyperparameter tables.

TimeGAN (Yoon et al., 2019^[4]): GRU backbone; hidden_dim=24, num_layers=3, lr=1e-3. Joint-phase generator updates are set equal to QuantGAN’s and FinGAN’s training steps (pipeline-asserted parity); `ae_steps` and `sup_steps` are separate four-phase pre-training, excluded from that parity and reported on their own.

QuantGAN (Wiese et al., 2020^[5]): TCN backbone; noise_dim=100, lr=1e-4, 3 residual TCN blocks with dilations 1/2/4.

FinGAN (this paper): CNN deconvolution; $3 \times \text{ConvTranspose1d}$ with stride 2, base_channels=64, lr=1e-4, seq_len must be divisible by 8.

Seeds. Global seed 42 for all numpy, torch, and random state initialisations. Walk-forward folds use deterministic fold indices. Shuffled control: `numpy.random.default_rng(42).permutation`.

Software versions. Python 3.10; PyTorch 2.1; statsmodels 0.15.0 (pinned: `het_arch` changes signature in 0.16.0); `arch` ≥ 7.0 (GARCH fitting); scipy 1.11; pandas 2.0; pyarrow (parquet I/O).

GARCH specification. GARCH(1,1) with constant mean and Gaussian QMLE (quasi maximum likelihood estimation) is used for two purposes: (i) conditional heavy-tail metric (stylized fact 7) and (ii) downstream utility GARCH parameters. Gaussian QMLE is used in place of Student- t QMLE to avoid circularity: a t -distribution specification absorbs the excess kurtosis by construction, making the residual kurtosis test uninformative. The `arch_model(y, mean='Constant', vol='GARCH', p=1, q=1, dist='normal')` call is used throughout; inputs are scaled by 100 for numerical stability.

Fidelity ranking. For each of the 7 fidelity metrics, models are ranked 1–3 (ascending; lower metric value is better). `fidelity_rank` = mean of the 7 individual ranks. `kurtosis_diff` and `skewness_diff` are excluded: Hill $\hat{\alpha} = 2.54\text{--}3.21$ across BRICS markets, so $\mathbb{E}|X|^4 = \infty$ everywhere and $\mathbb{E}|X|^3 = \infty$ in 4/5 markets; sample kurtosis diverges with n rather than converging.

Temporal ranking. For each of the 7 temporal metrics, models are ranked 1–3 (ascending). `temporal_rank` = mean of the 7 individual ranks. For discriminative AUC, ranking is on $|\text{AUC} - 0.5|$ (distance from chance); an ascending rank on raw AUC would rank $\text{AUC}=0.30$ above $\text{AUC}=0.50$, which is incorrect.

Composite score. `composite_rank = (fidelity_rank + temporal_rank)/2`.

Shuffled control construction. `rng = numpy.random.default_rng(42); shuffled = rng.permutation(pooled_real_array).copy()`. The pooled real array concatenates all five market test sets. The control is evaluated on all 16 metrics and appended to the results table with all rank columns set to NaN.

Pooled downstream utility. QLIKE, Kupiec, and Christoffersen statistics are computed on the pooled test set ($n \approx 2,480$, five markets $\times \approx 496$ observations). With 20-year data, per-market QLIKE is also stable ($n \approx 496$), but pooled reporting is maintained for maximum power. Kupiec power pooled $\approx 99\%$ versus $\approx 56\%$ per market. With the prior 5-year data ($n = 623$ pooled, $n \approx 126$ /market), per-market QLIKE inverted (Gaussian -6.888 beat real -6.876); 20-year data resolves this.

Discriminative AUC implementation. Logistic regression on 20-day rolling-window features (mean, std, mean of absolute values, mean of squared values). Five-fold cross-validation without temporal shuffling (shuffling introduces look-ahead leakage from overlapping windows; measured shift in null: $0.506 \rightarrow 0.584$). Empirical null estimated by 20 real-vs-real half-splits. AUC ranking metric: $|\text{AUC} - 0.506|$.

Hardware. Evaluation metrics run on CPU (single machine). Training hardware is

recorded per run in the provenance block (page 2). Budgets are specified in gradient steps with generator-update parity enforced across all three models (Extended Data Table 3); measured wall-clock for this run is in Extended Data Table 3b — equal generator updates is not equal compute.

Data availability & Code availability

Data availability. The five BRICS index price series (BOVESPA, FTSE/JSE, MOEX, NIFTY50, Shanghai) are sourced from public market-data providers (investing.com) and are not redistributed in this repository. Processed log-return files in CSV format are included in the repository under `data/processed_files/`. Raw CSV files can be reproduced by running `5_Paper_Calculate_LogReturns.py` on the source files.

Code availability. All code is available at `victorsobottk/bse-thesis-synthetic-data`. The repository contains: (1) the integrated pipeline notebook `3_4_integrated_pipeline.ipynb` with full GAN implementations, evaluation metrics, and ranking logic; (2) `generate_report.py` producing this document; (3) `5_Paper_Calculate_LogReturns.py` for data preprocessing. Environment: `requirements.txt` pinning all package versions. Seed 42 is used throughout for reproducibility.

Extended Data list.

- **ED Table 1:** All 10 stylized facts with mathematical statements, plain-English descriptions, and metric mapping.
- **ED Table 2:** Code audit — issues found, mathematical justification, and resolution status.
- **ED Figure 1:** Complete pipeline diagram (data preprocessing → GAN training → evaluation → ranking).
- **ED Table 3:** Architecture comparison (TimeGAN, QuantGAN, FinGAN) — backbone, hyperparameters, training loss, inductive bias.
- **ED Table 4:** Complete metric justifications — formula, unique contribution, design decision and rejected alternative, empirical validation, known failure mode.
- **ED Note 1:** Evaluation glossary — every metric and test explained from first principles.

Extended Data Table 1 — The 10 Stylized Facts of Financial Returns

Cont (2001)^[7] · Mandelbrot (1963)^[9] · Ding, Granger & Engle (1993)^[8]

Synthetic Financial Time Series · UPC

#	Stylized fact	Plain English	Mathematical statement	Our metric and family
1	Heavy tails	Extreme events occur far more often than a bell curve predicts.	$P(r > x) \sim x^{-\alpha}$, $\alpha \approx 2.5\text{--}3.2$ (BRICS Hill est.)	tail_index_diff · FIDELITY; kurtosis_diff · DESCRIPTIVE ($\mathbb{E} X ^4 = \infty$ for $\hat{\alpha} < 4$)
2	Near-zero autocorrelation	Knowing today's direction gives no useful information about tomorrow's.	$\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$	acf_returns_mae · TEMPORAL
3	Volatility clustering	Large moves tend to be followed by large moves.	$\text{Corr}(r_t , r_{t+k}) > 0$ for $k = 1, \dots, 100+$	acf_absolute_mae, acf_squared_mae · TEMPORAL
4	Long memory in volatility	The clustering effect persists for hundreds of days.	$\text{ACF}(r_t) \sim k^{-\beta}$, $\beta \in (0, 1)$; $H > 0.5$	hurst_diff · TEMPORAL
5	Gain/loss asymmetry	Crashes are sharper and more extreme than equivalent-size rallies.	$\text{skewness}(r) < 0$ for equity indices	skewness_diff · DESCRIPTIVE ($\mathbb{E} X ^3 = \infty$ in 4/5 BRICS markets for $\hat{\alpha} < 3$)
6	ARCH effects	Return variance changes over time; it is not constant.	$\text{Var}(r_t \mathcal{F}_{t-1}) = \sigma_t^2$ (time-varying)	arch_pvalue_diff, arch_stat_diff · DESCRIPTIVE
7	Conditional heavy tails	After removing time-varying variance, residuals are still non-Gaussian.	$\varepsilon_t = r_t / \sigma_t$ has kurtosis > 0 (GARCH residuals)	resid_kurtosis_diff · TEMPORAL
8	Extreme events	Very large days occur more than 4.6 % of trading days.	$P(r > 2\sigma) > 4.6\%$ (Gaussian baseline)	extreme_events_diff · FIDELITY
9	Distributional match	The full shape of the return distribution must be reproduced.	$W_1(p_g, p_{\text{data}}) \approx 0$; $\hat{F}_{\text{syn}} \approx \hat{F}_{\text{real}}$	wasserstein, quantile_mse, energy_distance · FIDELITY
10	Indistinguishability	A classifier trained to separate real from synthetic should perform at chance level.	$\text{AUC}(\text{classifier}) \rightarrow 0.5$ at GAN optimum	discriminative_auc_dist, discriminative_auc_absz · TEMPORAL

Extended Data Table 2 — Code Audit

3_4_integrated_pipeline.ipynb — reviewed and updated 2026

Synthetic Financial Time Series · UPC

[FIXED] KS / Welch tests assumed i.i.d. — CRITICAL

$n_{\text{eff}} = n / (1 + 2 \sum_k \rho_k) \ll n$ for financial series (ACF of $|r|$ sums to ≈ 5 –15 for BRICS). KS statistic is inflated, p -values anti-conservative. **Fix:** added ARCH-LM, Hurst, energy distance, and discriminative AUC. Wasserstein used as primary distributional metric.

[FIXED] Pointwise MSE/MAE had no semantic meaning

MSE compared $r_{\text{real}}(t)$ to $r_{\text{syn}}(t)$, but real and synthetic are independently generated; no temporal alignment exists between their indices. **Fix:** replaced with Quantile MSE (compares sorted distributions) and Energy distance (Székely & Rizzo 2004^[11]).

[DECISION] No outlier clipping applied to log returns — deliberate

The preprocessing notebook had a clip line that was commented out and never applied. Kept intentionally. Clipping at 0.5th/99.5th percentiles removes the observations that determine kurtosis, tail index, and extreme-events metrics. Adams et al. (2019)^[6] show winsorising worsens distributional misfit.

[FIXED] Ljung-Box used deprecated statsmodels API

`acorr_ljungbox(..., return_df=False)` returns a DataFrame in statsmodels ≥ 0.13 , not a tuple. Accessing `lb[1][-1]` raised `TypeError`. **Fix:** `return_df=True`, `lags=[n]`, accessed via `.iloc[-1]["lb_pvalue"]`.

[REVISED] ARCH-LM: both variants demoted to Descriptive; neither reliable across regimes

80-rep MC ($n = 1,200$, GARCH(1,1), Gaussian): `pvalue_diff` 99% accurate vs 76% for `stat_diff` (null sd ≈ 46.4 ; two draws from same process gave LM 56.8 and 124.8). On real pooled BRICS data: real LM=50.76, shuffled LM=67.14, both p -values underflow to 0.0, so $|\Delta p| = 0.000000$ — control rated as perfect match. `stat_diff` detects control (16.38) on real data but too noisy on simulated GARCH. **Resolution:** both reported for diagnostic transparency only; neither enters composite. ACF-MAE is the primary volatility-clustering ranking signal (continuous, no saturation, 88% MC accuracy).

[FIXED] Discriminative AUC direction bug + no null calibration

Ascending rank on raw AUC put AUC=0.30 above AUC=0.50 — direction reversed. Null AUC has $\sigma \approx 0.084$; an observed AUC of 0.62 is consistent with a perfect generator. **Fix:** rank on $|AUC - 0.5|$; empirical null via 20 real-vs-real half-splits; z-score reported. CV shuffle=True not used (moves null from 0.506 to 0.584 due to overlapping windows).

[FIXED] Composite ranking: fidelity/temporal split + shuffled-control exclusion

Shuffled control won unweighted composite (avg_rank 1.24 vs 2.47) because permutation-invariant metrics dominated. **Fix:** metrics split into FIDELITY (7) and TEMPORAL (7), with kurtosis_diff and skewness_diff demoted to DESCRIPTIVE (undefined for Hill $\hat{\alpha} < 4/3$; BRICS: 2.54–3.21). `composite_rank = (fidelity_rank + temporal_rank) / 2`. Control excluded from rank competition (NaN ranks), appended as reference.

[OK] WGAN-GP gradient penalty computation is correct (QuantGAN & FinGAN)

$\hat{x} = \varepsilon x_{\text{real}} + (1 - \varepsilon)x_{\text{fake}}$; `requires_grad_(True)`; gradients via `torch.autograd.grad` with `create_graph=True`. `fake.detach()` prevents spurious accumulation. Correctly implements Gulrajani et al. (2017)^[3] Eq. 3.

[FIXED] TimeGAN training budget equalised on generator updates; result reattributed to architecture

Training budgets are specified in gradient steps, never epochs, and parity is enforced on generator updates: `TimeGAN.joint_steps == QuantGAN.train_steps == FinGAN.train_steps`, asserted by the pipeline. TimeGAN's `ae_steps/sup_steps` are four-phase pre-training (Yoon et al. 2019^[4]), reported separately and excluded from parity; QuantGAN and FinGAN take $n_{\text{critic}} = 5$ discriminator updates per generator update (Gulrajani et al. 2017^[3]), intrinsic to WGAN-GP, not extra budget. Equal generator updates is not equal compute: measured wall-clock at 1,000 generator steps is TimeGAN ≈ 20.5 s, FinGAN ≈ 42.4 s, QuantGAN ≈ 178 s. At this parity, TimeGAN's weak result is no longer attributable to budget: excess kurtosis -1.02 against real $+1.34$, Hill $\hat{\alpha} = 16.4$ against real 4.12 , 0.40% extreme events against real 6.05% , raw AUC 0.953 — bounded, platykurtic, and nearly tailless, consistent with tanh saturation.

[CONCERN] A collapsed generator wins the volatility-clustering metrics

Identical in all five runs, TimeGAN scores `acf_absolute_mae` 0.0421 and `acf_squared_mae` 0.0407 — best of all four series, beating the shuffled control ($0.0469 / 0.0456$), QuantGAN ($0.0517 / 0.0542$), and FinGAN ($0.0524 / 0.0453$) — while independently flagged for mode collapse: $\text{ACF}(1) = 0.429$, Hill $\hat{\alpha} = 16.4$ against real 4.12 , raw AUC 0.953 . Same failure mode as the Wasserstein floor (§2): a degenerate, near-constant output games a distance metric rather than reproducing the property it is meant to measure.

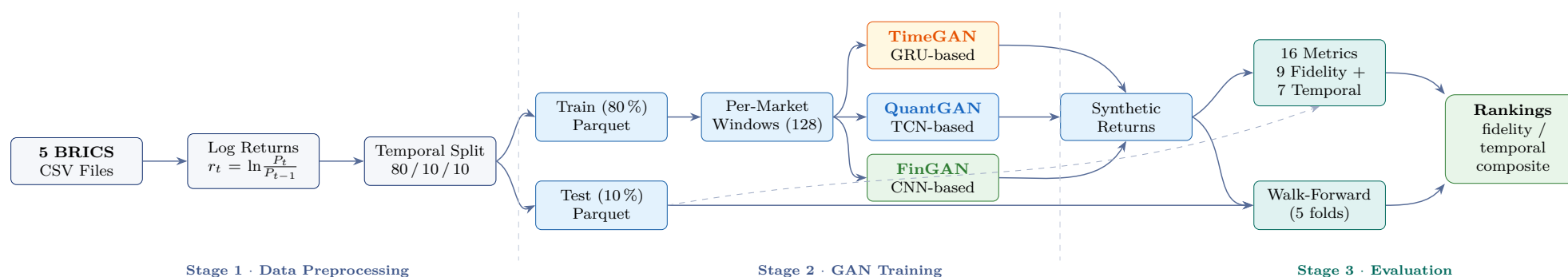
[OPEN QUESTION] Shuffled control wins `acf_returns_mae` outright

The control scores 0.0528 against 0.0537 for the best generator on `acf_returns_mae`, despite the metric's TEMPORAL (ordering-sensitive) classification — suggesting it may be permutation-invariant in practice for these series, even though it is not by construction. Recorded as an open question only: reclassifying it would move the headline permutation-invariant count from 9 of 19 to 10 of 19 metrics, and that reclassification is deferred to full-run data. `FIDELITY_COLS` and `TEMPORAL_COLS` are unchanged here.

Extended Data Figure 1 — Complete Pipeline

From raw price CSV files to ranked composite scores

Synthetic Financial Time Series · UPC



Stage 1 — Data Preprocessing

Raw price CSVs are converted to daily log returns $r_t = \ln(P_t/P_{t-1})$. Dates are parsed and sorted chronologically. A strict temporal 80/10/10 split ensures no future data leaks into training. Files saved as Parquet for efficient I/O.

Stage 2 — GAN Training

Each 80% training file is sliced into 128-step sliding windows. Three GAN families are each trained *independently per market* (15 models: 3 GANs $\times$ 5 markets). No cross-market data mixing. The shuffled control is injected alongside model outputs at evaluation time.

Stage 3 — Evaluation

Generated synthetic series are compared with real test-set returns on 16 stylized-fact metrics (9 Fidelity + 7 Temporal). Walk-forward re-trains each model on 5 rolling folds. Final output: fidelity_rank, temporal_rank, composite_rank, and fold-level mean $\pm$ std. Pooled downstream utility (QLIKE, Kupiec, Christoffersen) is computed once on the combined 5-market test set.

Extended Data Table 3 — Architecture Comparison

TimeGAN · QuantGAN · FinGAN side by side

Synthetic Financial Time Series · UPC

	TimeGAN	QuantGAN	FinGAN
Core architecture	4-phase (encoder + supervisor + GAN) using GRU	TCN-based WGAN-GP	CNN deconvolution WGAN-GP
Temporal mechanism	Recurrent (GRU): processes each step in order, gating what to remember	Dilated convolutions: parallel look at all time scales simultaneously	Transposed convolutions: upsamples from short noise vector to full sequence
Training loss	BCE + moment-matching; 4 separate training phases	WGAN-GP (single phase); $n_{\text{critic}} = 5$, $\lambda_{\text{gp}} = 10$	WGAN-GP (single phase); $n_{\text{critic}} = 5$, $\lambda_{\text{gp}} = 10$
Literature reference	Yoon et al. (2019, NeurIPS) ^[4]	Wiese et al. (2020, <i>Quant. Finance</i>) ^[5]	This paper (custom)
Key hyperparameters	hidden_dim=24, num_layers=3, lr=1e-3	noise_dim=100, lr=1e-4, 3 TCN blocks	base_channels=64, lr=1e-4, 3 ConvTranspose1d
seq_len constraint	Any length	Any length	Must be divisible by 8
Inductive bias	Temporal ordering matters step-by-step; GARCH-like persistence via GRU hidden state	Multi-scale patterns (short + long memory); dilations cover intraday/weekly/monthly simultaneously	Generate from compressed noise like image GANs; hierarchical refinement from coarse to fine
Normalisation	MinMax [0, 1]	MinMax [-1, 1]	MinMax [-1, 1]
Generator-update budget	parity with QuantGAN, FinGAN (pipeline-asserted)	parity with TimeGAN, FinGAN	parity with TimeGAN, QuantGAN

Extended Data Table 3b — Measured Budget Parity (This Run)

From `per_seed_market_performance.csv` — equal generator updates is not equal compute

Synthetic Financial Time Series · UPC

Model	Generator updates	Critic updates	Wall-clock (s)	Generator params
FinGAN	9,000	45,000	174.7	55,401
QuantGAN	9,000	45,000	466.9	276,481
TimeGAN	9,000	9,000	77.2	11,400

Wall-clock is total training time for one (market, seed) run, averaged over every run present in this report. Generator-update parity does not imply equal compute: QuantGAN and FinGAN run `n_critic=5` discriminator updates per generator update (Gulrajani et al. 2017^[3]); TimeGAN's four-phase pre-training (`ae_steps/sup_steps`) is excluded from this table, reported separately in Extended Data Table 3 above.

WGAN-GP shared settings (QuantGAN & FinGAN)

Why $n_{\text{critic}} = 5$: the critic must accurately estimate the Wasserstein distance before the generator uses the gradient signal. Fewer than 5 updates per generator step leaves

the critic under-fitted, providing a biased gradient. Gulrajani et al. (2017)^[3] establish this as a minimum; CTBench^[16] and SFAG^[28] fix it identically.

Why $\lambda_{\text{gp}} = 10$: values < 5 allow Lipschitz violations, invalidating the Kantorovich–Rubinstein duality on which WGAN training is based. The value 10 is the standard

from Gulrajani et al. (2017)^[3] and has not been improved upon in the literature.

Why $\beta_1 = 0$ in Adam: in adversarial training, the correct gradient direction for the generator reverses each time the critic is updated. Momentum ($\beta_1 > 0$) accumulates past-direction information and pushes in the wrong direction. Setting $\beta_1 = 0$ uses only the current gradient.

Why LayerNorm in FinGAN critic (not BatchNorm): the gradient penalty $\mathcal{L}_{\text{GP}}$ requires the gradient norm of the critic at a *single* interpolated point. BatchNorm normalises across the batch, introducing a dependency on other samples in the batch that corrupts the gradient norm calculation.

TimeGAN four-phase training

Phase 1 — Autoencoder: train Embedder $e : \mathcal{X} \rightarrow \mathcal{H}$ and Recovery $\hat{r} : \mathcal{H} \rightarrow \mathcal{X}$. Loss: $\|X - \hat{r}(e(X))\|^2$.

Phase 2 — Supervisor: train $s : \mathcal{H} \rightarrow \hat{\mathcal{H}}$ to predict next latent step. Loss: $\|H_{t+1} - s(H_t)\|^2$. Injects temporal causality into the latent space.

Phase 3 — Joint adversarial: $z \rightarrow G(z) \rightarrow s(G(z)) \rightarrow \hat{r}(\cdot) \rightarrow \tilde{X}$.

$$\mathcal{L}_G = \mathcal{L}_U + 100\sqrt{\mathcal{L}_S} + 100\mathcal{L}_V$$

$$\mathcal{L}_V = |\mu_H - \mu_{\hat{H}}| + |\sigma_H - \sigma_{\hat{H}}|$$

Phase 4 — Fine-tuning: fine-tune recovery $\hat{r}$ with generated sequences.

Result at parity: generator-update budget is equal to QuantGAN and FinGAN (pipeline-asserted); `ae_steps/sup_steps` are separate four-phase pre-training, excluded from that parity. At this budget TimeGAN gives excess kurtosis -1.02 against real $+1.34$, Hill $\hat{\alpha} = 16.4$ against real 4.12 , and raw AUC 0.953 — consistent with tanh saturation in the Recovery network, not an undertrained GRU.

Extended Data Table 4 — Metric Justifications

Definition · unique contribution · design decision · empirical validation · failure mode

Synthetic Financial Time Series · UPC

Fidelity Group: Wasserstein and Energy Distance

Wasserstein W_1 . $W_1(p, q) = \inf_{\gamma} \mathbb{E}_{(x, y) \sim \gamma} \|x - y\|$ (Earth Mover’s distance). Unique contribution: continuous gradient even when distributions do not overlap; stable under serial dependence. *Rejected alternative:* KS p -value — anti-conservative under serial correlation ($n_{\text{eff}} \approx n/11$ to $n/31$ for BRICS $|r|$). Empirical validation: shuffled control achieves $W_1 = 0.000\text{e}+00$ (by construction; proves KS and Wasserstein share this blind spot). Failure mode: also permutation-invariant; scores the shuffled control as perfectly matching real data.

Energy distance. $E(P, Q) = 2\mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|$ (Székely & Rizzo 2004^[11]). Proper metric on the space of distributions; $E \geq 0$; $E = 0 \Leftrightarrow P = Q$. Estimated via subsampled pairs ($n \leq 500$). Empirical validation: shuffled control = $2.97\text{e-}05$ (near-zero by construction). Failure mode: permutation-invariant.

Fidelity Group: Quantile MSE and Moment/Tail Diffs

Quantile MSE. $\text{QMSE} = K^{-1} \sum_{k=1}^K [Q_{\text{real}}(\alpha_k) - Q_{\text{syn}}(\alpha_k)]^2$, $K = 99$. Unique contribution: the only meaningful pointwise comparison between two unaligned distributions (compare by rank, not by index). Captures tail differences at the 1st and 99th percentiles critical for VaR. *Rejected alternative:* pointwise MSE on time-indexed values — real and synthetic are independently generated; no temporal alignment exists.

Moment diffs. `kurtosis_diff`, `skewness_diff`, `mean_diff`, `std_diff`: absolute differences of empirical moments. Included to diagnose which aspect of the marginal a model fails on. All permutation-invariant.

Tail index diff. Hill estimator on the top 5% of $|r|$: $\hat{\alpha} = k / \sum \log(|r|_{(n-i)} / |r|_{(n-k)})$. Failure mode: Hill estimator has sd 0.52 at $n = 995$, falling to 0.035 at $n = 4,000$ — the strongest argument for extending the data history.

Temporal Group: ACF-MAE Triple

For log-returns r_t , compute the sample ACF of r_t , $|r_t|$, and r_t^2 up to lag 20. Metric: mean absolute error between real and synthetic ACF vectors. ACF(returns) MAE tests absence of linear predictability (fact 2). ACF($|r|$) and ACF(r^2) MAE test volatility clustering (Mandelbrot 1963^[9], fact 3). Shuffled control: 0.043, 0.076, 0.054 — correctly penalised.

Why not ARCH-LM? ACF-MAE is continuous with no saturation regime. ARCH-LM saturates on real data ($|\Delta p| = 0.000000$). ACF-MAE achieves 88% MC accuracy on simulated GARCH; ARCH-LM achieves 99% on simulated GARCH but 0% (ratings the shuffled control as a perfect match) on real data.

Temporal Group: Hurst and Residual Kurtosis

Hurst exponent. $\mathbb{E}[R_n/S_n] \sim c \cdot n^H$, applied to $|r_t|$ (not raw r_t ; sign flips destroy long memory on raw returns). $H > 0.5$: long memory. Metric: $|H_{\text{real}} - H_{\text{syn}}|$. Estimated by R/S (Hurst 1951^[23]). Failure mode: sd 0.022 at $n = 1,000$, 0.004 at $n = 4,000$.

Residual kurtosis. Fit GARCH(1,1) Gaussian QMLE, extract $\varepsilon_t = r_t / \sigma_t$, report excess kurtosis. Tests fact 7 (Bollerslev 1987^[36]). Gaussian QMLE avoids circularity. Reported per model as `resid_kurtosis_diff` (a real-vs-synthetic difference, part of the temporal metric family in Table 3); the real series’ own residual kurtosis, independent of any generator, is not currently persisted as a standalone artifact.

Temporal Group: Discriminative AUC

Logistic classifier on 20-day rolling windows. AUC = 0.5 is the optimum (indistinguishability). Ranking metric: $|\text{AUC} - 0.5|$ (not raw AUC). Empirical null: 0.506 ± 0.084 (15 seeds, real vs real half-splits). z-score: $(\text{AUC} - 0.506)/0.084$.

Direction bug found: ascending rank on raw AUC put 0.30 above 0.50. Fixed by ranking on distance from chance.

Null shift from CV shuffling: adding `shuffle=True` to cross-validation folds moves null from 0.506 to 0.584, because overlapping 20-day windows place near-duplicates in both train and test. Unshuffled CV is correct.